
TP 2.1 - GENERADORES PSEUDOALEATORIOS

Juan Pablo Jaca

Legajo: 50311

Comisión: 402

Universidad Tecnológica Nacional - FRRO

Zeballos 1341, S2000, Argentina

juampijaca@gmail.com

Nahuel Berli

Legajo: 50310

Comisión: 402

Universidad Tecnológica Nacional - FRRO

Zeballos 1341, S2000, Argentina

nahuel.berli@gmail.com

Juan Cruz Mondino

Legajo: 51922

Comisión: 402

Universidad Tecnológica Nacional - FRRO

Zeballos 1341, S2000, Argentina

juancm.2000@hotmail.com

Gustavo Giampietro

Legajo: 50671

Comisión: 402

Universidad Tecnológica Nacional - FRRO

Zeballos 1341, S2000, Argentina

gustgiam2001@gmail.com

Mateo Seffino

Legajo: 50939

Comisión: 402

Universidad Tecnológica Nacional - FRRO

Zeballos 1341, S2000, Argentina

seffinomateo@gmail.com

Alexis Mateo

Legajo: 51191

Comisión: 402

Universidad Tecnológica Nacional - FRRO

Zeballos 1341, S2000, Argentina

alexisjoelmateo@gmail.com

14 de mayo de 2025

ABSTRACT

El siguiente documento tiene por objetivo presentar el trabajo de investigación realizado para introducir en uno de los elementos fundamentales para gran parte de las simulaciones, los generadores de números pseudoaleatorios.

Keywords Simulación · generadores pseudoaleatorios

Índice

1. Introducción	4
2. Generadores de Números Aleatorios Reales	4
3. Generadores de Números Pseudoaleatorios	4
3.1. Pruebas de Aleatoriedad	5
3.2. Generador Lineal Congruencial (GLC)	5
3.3. Generador de Multiplicación con Acarreo (MWC)	6
3.4. Generador Mersenne Twister (MT)	8
3.5. Generador Itamaracá (ITA)	8
4. Tests aplicados a los generadores para la evaluación de su calidad	10
4.1. Tests de Ausencia de Patrones	10
4.1.1. Test de Dispersión (Scatter Plot)	10
4.1.2. Test Serial	10
4.2. Tests de Independencia	10
4.2.1. Test de Autocorrelación	10
4.3. Tests de Uniformidad	10
4.3.1. Test de Chi-Cuadrado	11
4.3.2. Test de Kolmogorov-Smirnov	11
4.3.3. Test de la Media	11
4.4. Resultados de los Tests	11
5. Metodología de Trabajo	12
5.1. Implementación de los generadores	12
5.2. Visualización con imágenes de ruido	13
5.3. Tests estadísticos	13
5.4. Ejecución y estructura del código	13
5.5. Bibliotecas utilizadas	13
6. Gráficas	14
6.1. Generador Lineal Congruencial (GLC)	14
6.1.1. Gráfico de Ruido	14
6.1.2. Scatter Plot en el Test de Dispersión	15
6.1.3. Histograma 2D en el Test Serial	16
6.2. Generador de Multiplicación con Acarreo (MWC)	17
6.2.1. Gráfico de Ruido	17
6.2.2. Scatter Plot en el Test de Dispersión	18
6.2.3. Histograma 2D en el Test Serial	18
6.3. Generador Mersenne Twister	19

6.3.1.	Gráfico de Ruido	19
6.3.2.	Scatter Plot en el Test de Dispersión	19
6.3.3.	Histograma 2D en el Test Serial	20
6.4.	Generador Itamaracá (ITA)	20
6.4.1.	Gráfico de Ruido	20
6.4.2.	Scatter Plot en el Test de Dispersión	21
6.4.3.	Histograma 2D en el Test Serial	22
7.	Fórmulas empleadas	22
7.1.	Fórmulas empleadas por los Generadores de Números Pseudoaleatorios	22
7.1.1.	Generador Lineal Congruencial (GLC)	22
7.1.2.	Generador de Multiplicación con Acarreo (MWC)	23
7.1.3.	Generador Mersenne Twister (MT)	23
7.1.4.	Generador Itamaracá (ITA)	24
7.1.5.	Normalización al intervalo [0, 1)	25
7.2.	Fórmulas empleadas por los Tests de los Generadores	25
7.2.1.	Test de Dispersión (Scatter Plot)	25
7.2.2.	Test Serial	26
7.2.3.	Test de Autocorrelación	26
7.2.4.	Test Chi-Cuadrado	27
7.2.5.	Test Kolmogorov-Smirnov	28
7.2.6.	Test de la Media	28
8.	Conclusiones	29
	Referencias	29

1. Introducción

Un número pseudoaleatorio es un valor numérico generado mediante un proceso computacional que, aunque parece producir resultados al azar, en realidad no lo hace de forma genuina. Se trata de secuencias de números que, desde un punto de vista estadístico, no presentan patrones evidentes ni regularidades fácilmente detectables, lo que las hace útiles para una amplia variedad de aplicaciones donde se requiere un comportamiento aleatorio. Sin embargo, estos números no son verdaderamente aleatorios, ya que se originan a partir de algoritmos deterministas que, dados los mismos parámetros iniciales (como una semilla o estado inicial), generan exactamente la misma secuencia de salida. Esta propiedad de reproducibilidad puede ser tanto una ventaja —por ejemplo, en simulaciones científicas que requieren repetibilidad— como una limitación, especialmente en contextos que exigen un alto grado de imprevisibilidad, como la criptografía.

El estudio de los números pseudoaleatorios abarca tanto el diseño y análisis de algoritmos generadores como la evaluación estadística de su calidad. Existen múltiples tipos de generadores pseudoaleatorios, entre los cuales se destacan los generadores lineales congruenciales (GLC) y los de multiplicación con acarreo (MWC), que serán abordados en este informe. Los GLC son de los más antiguos y sencillos de implementar, basados en una relación recursiva modular, mientras que los MWC pertenecen a una familia más moderna que introduce mecanismos adicionales como el “acarreo” para mejorar la distribución y reducir correlaciones. Evaluar la calidad de estas secuencias implica aplicar una serie de pruebas estadísticas que buscan detectar patrones o correlaciones no deseadas, asegurando que el generador pueda ser utilizado con confianza en contextos de simulaciones, modelado estocástico, videojuegos, inteligencia artificial, y otras áreas donde la aleatoriedad computacional es fundamental. Este análisis es crucial, ya que incluso pequeñas desviaciones respecto a la aleatoriedad ideal pueden afectar los resultados de los modelos o experimentos que dependen de estas secuencias.

2. Generadores de Números Aleatorios Reales

Un **número aleatorio** es un número seleccionado de un conjunto de números limitados o ilimitados que no sigue un patrón de predicción discernible. Este conjunto de números casi siempre es independiente entre sí. Sin embargo, puede seguir una distribución específica. Por ejemplo, la estatura de los alumnos de una escuela tiende a seguir una distribución normal en torno a la mediana. Si la estatura de un alumno se selecciona al azar, el número seleccionado tiene mayor probabilidad de estar más cerca de la mediana que de ser clasificado como muy alto o muy bajo. Los generadores de números aleatorios anteriores asumen que los números generados son independientes entre sí y se distribuirán uniformemente en todo el rango de valores posibles.

Un generador de números aleatorios, como los mencionados anteriormente, es un dispositivo que puede generar uno o varios números aleatorios dentro de un rango definido. Los números verdaderamente aleatorios se basan en fenómenos físicos como el ruido atmosférico, el ruido térmico y otros fenómenos cuánticos. Los métodos que generan números verdaderamente aleatorios también implican la compensación de posibles sesgos causados por el proceso de medición.

La desventaja de éstos métodos es que son costosos, tardados y no reproducibles.

3. Generadores de Números Pseudoaleatorios

Un **generador de números pseudoaleatorios** es un algoritmo para generar una secuencia de números cuyas propiedades se aproximan a las de las secuencias de números aleatorios. Los generadores informáticos de números aleatorios casi siempre son generadores de números pseudoaleatorios. Sin embargo, los números generados por estos generadores no son verdaderamente aleatorios. Los números aleatorios generados son suficientes para la mayoría de las aplicaciones, pero no deben utilizarse con fines criptográficos.

Los números pseudoaleatorios se generan de manera secuencial con un algoritmo determinístico, formalmente se definen por:

- Función de inicialización: recibe un número (la semilla) y pone al generador en su estado inicial.
- Función de transición: transforma el estado del generador.
- Función de salida: transforma el estado para producir un número fijo de bits (0 ó 1).

Una sucesión de bits pseudoaleatorios se obtiene definiendo la semilla y llamando repetidamente a la función de transición y a la función de salida.

Esto implica, entre otras cosas, que una sucesión de números pseudoaleatorios esta completamente determinada por la semilla.

Se busca que en una secuencia de números pseudoaleatorios:

- No se muestre ningún patrón o regularidad aparente desde un punto de vista estadístico.
- Dada una semilla inicial, se puedan generar muchos valores antes de repetir el ciclo.

3.1. Pruebas de Aleatoriedad

Para evaluar la calidad de los generadores de números aleatorios existen dos tipos de pruebas:

- **Empíricas:** evalúan estadísticas de sucesiones de números.
- **Teóricas:** se establecen las características de las sucesiones usando métodos de teoría de números con base en la regla de recurrencia que generó la sucesión.

La información para la Sección 2 y la Sección 3 (hasta este punto) fue obtenida de (Calculator.net, 2025) y (Tereom, 2018).

3.2. Generador Lineal Congruencial (GLC)

Un **Generador Lineal Congruencial (GLC)** es un algoritmo que permite obtener una secuencia de números pseudoaleatorios calculados con una función lineal definida a trozos discontinua. Es uno de los métodos más antiguos y conocidos para la generación de números pseudoaleatorios. La teoría que sustenta el proceso es relativamente fácil de entender, el algoritmo en sí es de fácil implementación y su ejecución es rápida.

Definición

La secuencia de números enteros $X_1, X_2, X_3, \dots$ está definida por la **relación de recurrencia**:

$$X_i = (aX_{i-1} + c) \text{ mód } m$$

con $0 \leq X_i \leq m - 1$, donde m (el módulo), a (el multiplicador), c (el incremento) y X_0 (la semilla o valor inicial) son números enteros no negativos.

Además, a los números m, a, c y X_0 se les impone las condiciones

$$m > 0, \quad 0 < a < m, \quad 0 \leq c < m \quad \text{y} \quad 0 \leq X_0 < m.$$

Si $c = 0$, el generador es llamado frecuentemente **Generador Congruencial Lineal Multiplicativo (GLMC)** o generador de números pseudoaleatorios de Lehmer, pero si $c \neq 0$, el generador es llamado **Generador Congruencial Lineal Mixto (GLMC)**.

Cada entero X_i queda completamente determinado por las constantes m, a, c y X_0 , pues puede demostrarse por *inducción matemática* que para $i = 1, 2, 3, \dots$:

$$X_i = \left[a^i X_0 + \frac{c(a^i - 1)}{a - 1} \right] \text{ mód } m$$

Periodo

Se dice que un generador congruencial cumple un ciclo cuando el número entero X_i , con $1 \leq i \leq m$, coincide con la semilla X_0 , cuando sucede esto, la secuencia de números generados se repetirá en el mismo orden.

El **período** de un generador congruencial se define como la longitud de un ciclo y dado que X_i depende de X_{i-1} y $0 \leq X_i \leq m-1$, entonces el período máximo puede ser a lo más su módulo m y en tal caso diremos que el generador congruencial tiene un **período completo**.

Si un generador congruencial tiene un período completo entonces para cualquier valor que tome la semilla X_0 en $\{0, 1, \dots, m-1\}$ producirá un ciclo en algún orden, pero si el generador no tiene un período completo, esto es, su período es menor a m , entonces la longitud del ciclo dependerá del valor escogido de la semilla X_0 .

Teorema

El generador congruencial lineal dado por

$$X_i = (aX_{i-1} + c) \pmod{m}$$

tendrá período completo m para cualquier valor de la semilla X_0 en $\{0, 1, \dots, m-1\}$ si y sólo si:

1. c y m son primos relativos, es decir, $\text{mcd}(c, m) = 1$.
2. $a - 1$ es múltiplo de todos los factores primos de m , es decir, $a \equiv 1 \pmod{q}$ para todo q factor primo de m .
3. Si m es múltiplo de 4 entonces $a - 1$ también lo es, es decir, $m \equiv 0 \pmod{4} \iff a \equiv 1 \pmod{4}$.

Estas tres condiciones son conocidas como el **Teorema Hull-Dobell**.

El Generador Congruencial Lineal Multiplicativo (GLMC) no cumple la primera condición, pues $\text{mcd}(0, m) = m$, por lo que no tienen período completo m .

Aunque los GLC son capaces de producir números pseudoaleatorios que pueden pasar las pruebas de aleatoriedad, esta posibilidad es extremadamente sensible a la elección de los parámetros m , a y c .

Ventajas y Desventajas

- Son rápidos, y requieren poca memoria para retener su estado
- No deberían ser usados en aplicaciones para las que se requiera aleatoriedad de alta calidad.
- No deberían usarse para aplicaciones criptográficas

La información relacionada a este generador de números pseudoaleatorios GLC fue extraída de (Wikipedia contributors, 2025a).

3.3. Generador de Multiplicación con Acarreo (MWC)

En informática, la **Multiplicación con Acarreo (MWC)** es un método inventado por George Marsaglia para generar secuencias de enteros aleatorios a partir de un conjunto inicial de dos a miles de valores semilla elegidos aleatoriamente. Las principales ventajas del método MWC son que utiliza aritmética de enteros simple y permite la generación muy rápida de secuencias de números aleatorios con periodos inmensos, que varían de aproximadamente 2^{60} a $2^{20000000}$.

Al igual que con todos los generadores de números pseudoaleatorios, las secuencias resultantes son funciones de los valores de semilla suministrados.

Teoría General

Una secuencia MWC está basada en un módulo de aritmética en base b , usualmente $b = 2^{32}$, dado que dicho módulo en base b es automático en muchas computadoras. Sin embargo, a veces es usada una base $b = 2^{32} - 1$, porque la aritmética para módulos $2^{32} - 1$ requiere solamente un ajuste simple hacia 2^{32} , y la teoría para las secuencia MWC basadas en módulos 2^{32} presenta algunas dificultades muy molestas que pueden ser evitadas usando $b = 2^{32} - 1$. En su forma más común, un generador lag- r MWC requiere una **base** b , un **multiplicador** a , y un conjunto de $r + 1$ valores **semilla** aleatorios, consistente en r residuos de b ,

$$x_0, x_1, x_2, \dots, x_{r-1},$$

y un **acarreo** inicial $c_{r-1} < a$.

La secuencia lag- r MWC es entonces una secuencia de pares x_n, c_n determinada por

$$x_n = (ax_{n-r} + c_{n-1}) \bmod b, \quad c_n = \left\lfloor \frac{ax_{n-r} + c_{n-1}}{b} \right\rfloor, \quad n \geq r,$$

y la salida del generador MWC es la secuencia de x 's,

$$x_r, x_{r+1}, x_{r+2}, \dots$$

El período de un generador lag- r MWC es el orden de b en el grupo multiplicativo de números módulo $ab^r - 1$. Es habitual escoger a tal que $p = ab^r - 1$ es un primo para el cual el orden de b puede ser determinado. Ya que 2 es un **residuo cuadrático** de números de la forma $8k \pm 1$, $b = 2^{32}$ no puede ser una raíz primitiva de $p = ab^r - 1$. Por consiguiente no hay generadores MWC en base 2^{32} que presentan el período máximo posible, una de las dificultades que el uso de $b = 2^{32} - 1$ supera.

Un problema teórico con generadores MWC, indicado por Couture e l'Ecuyer (1997) es que los bits más significativos son levemente sesgados; los **generadores de multiplicación con acarreo complementaria** no comparten este problema: "Podemos ver que, para el MWC complementario, cada bit del valor de salida es justo, esto es, los dos dígitos binarios aparecerán con igual frecuencia en un período completo, una propiedad no compartida por generadores MWC." Los generadores de multiplicación con acarreo complementarios también requieren considerablemente más tiempo de cómputo por iteración, de tal manera que existe un compromiso de evaluar en dependencia de los requisitos de la implementación.

Comparación con generadores de congruencia lineales

Los **generadores de congruencia lineales** son implementados como

$$x_{n+1} = (ax_n + c) \bmod 2^{32}, \quad (1)$$

ya que la mayoría de los procesadores aritméticos son capaces de poner el multiplicador a y el x actual en registros de 32-bit, formando el producto en registros contiguos de 64 bits, y tomando los 32 bits inferiores como el producto, o sea, de forma

$$a \times x \bmod 2^{32}. \quad (2)$$

Agregando el c de 32-bit a esa mitad inferior se obtiene $(ax + c) \bmod 2^{32}$. Si $a \bmod 8$ es 1 o 5 y c es impar, la secuencia congruente resultante base 2^{32} tendrá un período 2^{32} .

Un generador lag-1 de multiplicación con acarreo nos permite hacer el período aproximadamente 2^{63} usando esas mismas operaciones computacionales, excepto que esta vez la mitad superior del producto de 64 bits es usada en vez de ignorada, después de que los 64 bits son formados. Es usado como un **nuevo** valor de acarreo c en vez del valor fijo de acarreo de la secuencia de congruencial estándar: tomando $ax + c$ en 64-bits, luego formando un nuevo c con la mitad superior de esos 64 bits, y el nuevo x con la mitad inferior.

Con el multiplicador a especificado, cada par de valores de entrada x, c es convertido en un nuevo par,

$$x \leftarrow (ax + c) \bmod 2^{32}, \quad c \leftarrow \left\lfloor \frac{ax + c}{2^{32}} \right\rfloor. \quad (3)$$

Si x y c no son cero ambos, entonces el período de la secuencia de multiplicación con acarreo resultante será de orden $b = 2^{32}$ en el grupo multiplicativo de residuales módulo $ab - 1$, o sea, en menor n tal que $b^n = 1 \pmod{ab - 1}$. Si escogemos a entre 28 y 31 bits tal que $ab - 1$ es un “primo seguro”, que ambos $ab - 1$ y $ab/2 - 1$ son primos, entonces el período será $ab/2 - 1$, acercándose a 2^{63} , lo cual en la práctica puede ser un subconjunto aceptablemente grande de posibles pares (x, c) de números de 32.

La información relacionada al generador de números pseudoaleatorios MWC fue obtenida de (Wikipedia contributors, 2025b).

3.4. Generador Mersenne Twister (MT)

El Mersenne Twister (MT) es uno de los generadores de números pseudoaleatorios más utilizados. Fue desarrollado en 1997 por Takuji Nishimura y Makoto Matsumoto. Su nombre proviene de su extenso período de $(2^{219937}) - 1$, que es un primo de Mersenne, un tipo de número primo de la forma $M_n = (2^n) - 1$, estudiado por el matemático francés Marin Mersenne en el siglo XVII. Este generador utiliza una semilla de 19937 bits que se almacena en un arreglo de 624 elementos, cada uno con valores de 32 bits.

El algoritmo destaca por su larga periodicidad y una equidistribución en 623 dimensiones con hasta 32 bits de precisión. Además, es eficiente tanto en términos de tiempo de cómputo —gracias a que emplea operaciones binarias— como de memoria, ya que solo requiere 624 palabras de almacenamiento. Esto lo hace ideal para muchas aplicaciones que requieren secuencias largas y aparentemente aleatorias de números.

Sin embargo, el Mersenne Twister presenta algunas limitaciones. Es predecible si se conoce su estado interno, lo que permite reconstruir valores pasados y anticipar los futuros. Su estructura es rígida y pesada, y a pesar de su tamaño de estado, no supera todos los tests estadísticos estándar. Por estas razones, en aplicaciones que demandan alta seguridad o paralelización eficiente, se prefieren generadores alternativos como los Counter-Based RNG (CBRNG), por ejemplo, Threefry o Threefish, que son más adecuados para sistemas paralelos con múltiples hilos o GPUs.

El funcionamiento del Mersenne Twister se basa en dos procesos clave: la inicialización del estado a partir de una semilla, y la generación de números, donde se extraen enteros del estado, se aplican desplazamientos de bits y se producen los valores aleatorios. Una vez consumidos los 624 enteros, el algoritmo ejecuta una operación llamada twisting, que reorganiza el estado para seguir generando números.

3.5. Generador Itamaracá (ITA)

Itamaracá o simplemente “**ITA**” es una nueva, sencilla y rápida base matemática en generadores de números pseudoaleatorios (PNRG) que genera “infinitas” secuencias de números en un rango $[0, 1]$ llevando a cabo una distribución uniforme.

El origen de su nombre proviene tupiguaraní en la que se refiere a “piedra que canta”, en este sentido una referencia a la casualidad, a un acontecimiento inesperado.

¿Cómo funciona?

Como todo GNPA (Generador de Números Pseudoaleatorios) ITA tiene algunas características distintivas. A continuación se presentan sus condiciones iniciales:

- En primer lugar, elegir N , es decir, un valor máximo dentro de un rango entre 0 y N seleccionado por un criterio elegido por el usuario, donde N pertenece a los naturales.
- En este modelo, hay tres semillas S_0 , S_1 y S_2 . Para cada una de estas semillas, elija un número cualquiera perteneciente a los naturales que pertenezca al intervalo entre 0 y N .

Tras elegir arbitrariamente los tres valores de las semillas, el proceso de cálculo se divide en dos etapas principales:

- Pn (Proceso de n) o Estado Intermedio
- Cálculo Final o Fórmula General

Pn (Proceso de n) o Estado Intermedio

En esta etapa, tenemos que tener en cuenta los valores absolutos que tienen las diferencias entre las dos semillas que son “móviles” en el tiempo, preferiblemente diciendo, en secuencia.

$$P_n = ABS(S_2 - S_0)$$

La información relacionada al generador de números pseudoaleatorios Itamaracá (ITA) fue extraída de (Pereira, 2022).

Cálculo Final o Fórmula General

En este paso, debemos multiplicar “x” resultado el obtenido en el primer paso (en Pn) por Xrn, es decir, cualquier valor deseado por el usuario, siempre que este valor sea muy cercano a 2 (ej.: 1.97, 1.98, 1.99789...).

$$FRNS = ABS[N - (P_n \cdot X_{rn})]$$

Ejemplo

Supongamos que queremos generar números del 0 al 10000:

N	10,000
Semilla 0	8,777
Semilla 1	11
Semilla 2	8

Tabla 1: Valores de las semillas

Podemos generar el **primer número** utilizando el estado intermedio (Pn) y luego utilizar la fórmula general, como se demuestra a continuación:

$$P_1 = ABS(8 - 8.777) = 8.769$$

$$FRNS_1 = ABS[10000 - (8.769 \cdot 1.97) = 7.275$$

Segundo número:

$$P_2 = ABS(7.275 - 11) = 7.264$$

$$FRNS_2 = ABS[10000 - (7.264 \cdot 1.97) = 4.310$$

Tercer número:

$$P_3 = ABS(4.310 - 8) = 4.302$$

$$FRNS_3 = ABS[10000 - (4.302 \cdot 1.97) = 1.525$$

Así, obtenemos los tres primeros números generados: 7.275, 4.310 y 1.525...

Los siguientes números generados por esta secuencia seguirán la misma lógica.

4. Tests aplicados a los generadores para la evaluación de su calidad

4.1. Tests de Ausencia de Patrones

El propósito de estas pruebas es determinar si un PRNG produce secuencias que carecen de regularidades, simetrías o repeticiones sistemáticas. La detección de patrones predecibles puede comprometer la calidad de la aleatoriedad, por lo que estos tests se enfocan en identificar dependencias internas o estructuras ocultas que sugieran un comportamiento no aleatorio. Entre las pruebas más representativas en esta categoría se analizan el test de dispersión y el test serial.

4.1.1. Test de Dispersión (Scatter Plot)

Esta técnica gráfica, conocida también como scatter plot en inglés, permite visualizar la relación entre dos variables numéricas mediante puntos en un plano cartesiano. Cada punto representa un par de valores correspondientes a las variables en estudio. Por ejemplo, al analizar la relación entre horas de estudio y calificaciones, se asigna a cada individuo un punto cuyas coordenadas reflejan ambas medidas.

Este tipo de representación facilita la identificación de patrones o tendencias. Una alineación más o menos lineal de los puntos indica correlación: positiva si ambos valores aumentan conjuntamente, o negativa si uno aumenta mientras el otro disminuye. Por el contrario, una distribución dispersa y sin forma aparente sugiere independencia entre las variables.

4.1.2. Test Serial

Esta prueba analiza la distribución de bloques consecutivos de elementos generados por el PRNG, en lugar de examinar valores individuales. El objetivo es verificar si la frecuencia con que aparecen pares, tríos u otros bloques de longitud fija se ajusta a la esperada en una secuencia aleatoria ideal.

4.2. Tests de Independencia

Asegurar que los valores generados sean mutuamente independientes constituye un aspecto esencial de la aleatoriedad. Las siguientes pruebas están orientadas a verificar que no existan correlaciones entre los elementos de la secuencia, es decir, que el valor de un número no dependa ni influya sobre los demás.

4.2.1. Test de Autocorrelación

Esta prueba examina la independencia entre valores separados por una distancia fija dentro de la secuencia generada. Su propósito es determinar si existe alguna relación predecible entre ellos. En una secuencia verdaderamente aleatoria, tales correlaciones deberían ser inexistentes, sin importar la separación entre elementos.

La detección de autocorrelación sugiere la presencia de dependencia interna, lo cual puede llevar a resultados predecibles o sesgados. Aunque esta prueba no detecta todos los posibles defectos de un generador, sí proporciona una herramienta eficaz para identificar relaciones lineales ocultas que comprometan la independencia estadística de la secuencia.

4.3. Tests de Uniformidad

Estas pruebas evalúan si los valores generados por el PRNG están distribuidos uniformemente en el intervalo previsto. Una distribución uniforme implica que cada subintervalo tiene la misma probabilidad de contener un valor generado, sin acumulaciones o vacíos sistemáticos. A continuación, se describen tres de las pruebas más utilizadas en esta categoría.

4.3.1. Test de Chi-Cuadrado

Esta prueba contrasta la distribución observada de los datos con una distribución uniforme teórica. Para ello, el intervalo se divide en clases y se compara la frecuencia observada en cada una con la esperada. Diferencias significativas entre ambas frecuencias indican que el generador puede estar sesgado.

El test de chi-cuadrado es una prueba global de bondad de ajuste y resulta especialmente eficaz para detectar concentraciones anómalas en ciertas regiones del intervalo. Sin embargo, su sensibilidad depende en gran medida del número de clases elegido y del tamaño de la muestra.

4.3.2. Test de Kolmogorov-Smirnov

A diferencia del chi-cuadrado, esta prueba no requiere segmentar el intervalo en clases. Compara directamente la función de distribución acumulada empírica con la teórica de una distribución uniforme, evaluando la máxima desviación entre ambas.

Este enfoque proporciona una evaluación más sensible a desviaciones tanto en la forma como en la localización de la distribución. Aunque su potencia aumenta con el tamaño de la muestra, el test de Kolmogorov-Smirnov es considerado un estándar riguroso para verificar la uniformidad en datos continuos e independientes.

4.3.3. Test de la Media

Partiendo de la premisa de que una distribución uniforme tiene un valor medio teórico conocido, esta prueba verifica si el promedio de una muestra de números generados se aproxima a ese valor. Por ejemplo, en el intervalo $[0, 1]$, el valor esperado es 0.5.

Se trata de una prueba puntual que evalúa una única propiedad estadística. Es útil para detectar desviaciones sistemáticas en el promedio, pero limitada frente a otros tipos de irregularidades, como acumulaciones localizadas o distribuciones asimétricas dentro del intervalo.

4.4. Resultados de los Tests

```
***** Tests: GCL *****
Chi_square_test:  True
Serial_test:     True
Kolmogorov_smirnov_test:  True
Media_test:     True
Autocorrelacion_test:  True
```

```
***** Tests: MWC *****
Chi_square_test:  True
Serial_test:     True
Kolmogorov_smirnov_test:  True
Media_test:     True
Autocorrelacion_test:  True
```

```
***** Tests: Marsenne Twister *****
Chi_square_test:  True
Serial_test:     True
Kolmogorov_smirnov_test:  True
Media_test:     True
Autocorrelacion_test:  True
```

```

***** Tests: Ita 1.9789 *****
Chi_square_test:  True
Serial_test:      True
Kolmogorov_smirnov_test:  True
Media_test:       True
Autocorrelacion_test:  True

***** Tests: Ita 2 *****
Chi_square_test:  False
Serial_test:      False
Kolmogorov_smirnov_test:  False
Media_test:       False
Autocorrelacion_test:  True

```

Se puede ver que para cada generador, se listan los tests ejecutados en los cuales se indica con True si los pasaron correctamente o False si no lo hicieron.

A su vez, tenemos las dos versiones de Ita, en las cuales el número seguido del nombre abreviado del generador es el valor elegido para x .

5. Metodología de Trabajo

En este trabajo, desarrollamos un conjunto de scripts en Python con el objetivo de estudiar la calidad de distintos generadores de números pseudoaleatorios. Con ese objetivo, implementamos distintos generadores, realizamos tests estadísticos sobre las secuencias generadas y generamos imágenes de ruido que permiten visualizar de forma intuitiva la distribución de los números obtenidos. El código fue organizado de forma modular, lo que favorece su claridad, reutilización y mantenimiento.

5.1. Implementación de los generadores

Se definieron cuatro clases correspondientes a distintos algoritmos generadores:

- **GCL (Generador Congruencial Lineal)**: basado en la fórmula $X_{n+1} = (aX_n + c) \bmod m$.
- **MWC (Multiply With Carry)**: utiliza una semilla y un acarreo para generar nuevos valores.
- **Mersenne Twister**: hace uso del generador incluido en la biblioteca estándar de Python, restringido a un módulo específico.
- **Ita (Itamaracá)**: un generador alternativo que combina tres semillas mediante una fórmula basada en diferencias y un factor multiplicativo.

Cada generador implementa los métodos `next()`, `get_random_number()` y `name()`, que permiten obtener números pseudoaleatorios en el intervalo $[0, 1)$ e identificar el tipo de generador.

5.2. Visualización con imágenes de ruido

Para complementar el análisis cuantitativo, se generaron imágenes en escala de grises de tamaño 256×256 , donde cada píxel se asigna a partir de un número pseudoaleatorio. Esto permite visualizar si existen patrones repetitivos o concentraciones no uniformes, indicadores de baja calidad en la aleatoriedad. Las imágenes fueron generadas con la función `generar_imagen_ruido()` utilizando la biblioteca `matplotlib`, y almacenadas en archivos `.png` con el nombre del generador correspondiente.

5.3. Tests estadísticos

Para evaluar rigurosamente la calidad de cada generador, se aplicaron los siguientes tests estadísticos sobre 1000 valores generados:

- **Chi-cuadrado:** compara la distribución de frecuencias con la distribución uniforme teórica.
- **Kolmogorov-Smirnov:** verifica si los números generados siguen una distribución uniforme continua.
- **Media:** contrasta la media muestral con la esperada (0.5) bajo hipótesis de uniformidad.
- **Autocorrelación:** mide la dependencia entre valores consecutivos mediante un test z.
- **Serial:** evalúa la independencia mediante un histograma bidimensional.
- **Scatter Plot:** genera un gráfico de dispersión entre pares consecutivos (x_n, x_{n+1}) , para detectar correlaciones visuales.

Todos los resultados se muestran por consola, e incluyen tanto valores de prueba como decisiones de aceptación o rechazo bajo un nivel de significancia $\alpha = 0.05$.

5.4. Ejecución y estructura del código

El script `generadores_pseudoaleatorios.py` contiene las clases de los generadores y define una lista con las instancias utilizadas. Los scripts `imagen.py` y `tests.py` importan esta lista y procesan automáticamente cada generador, ya sea generando la imagen correspondiente o ejecutando los tests estadísticos sobre la secuencia generada.

Ejemplo de ejecución:

Para ejecutar los tests y visualizar los resultados, basta con ejecutar en consola:

```
python tests.py
```

Para generar las imágenes de ruido de cada generador:

```
python imagen.py
```

5.5. Bibliotecas utilizadas

Las bibliotecas utilizadas en la implementación fueron:

- `numpy`: para cálculos numéricos y generación de arreglos.
- `matplotlib.pyplot`: para graficar imágenes y resultados de tests.
- `scipy.stats`: para acceder a funciones estadísticas como pruebas de hipótesis y distribuciones teóricas.

Para el desarrollo de los generadores pseudoaleatorios, además de las fuentes mencionadas anteriormente en este informe, se extrajo información de (QuantStart, 2025).

Por otro lado, para la implementación de los tests realizados sobre los generadores pseudoaleatorios, se recurrió a (UniverTools, 2017), (Watermarkero, 2017), (Pulido Cayuela, 2009) y (DilliHangrae, 2024).

6. Gráficas

6.1. Generador Lineal Congruencial (GLC)

6.1.1. Gráfico de Ruido

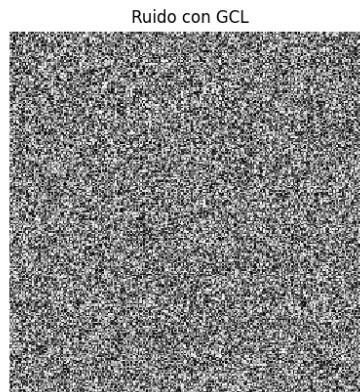


Figura 1: Gráfico de Ruido correspondiente al Generador Pseudoaleatorio GLC

Este gráfico corresponde a una visualización bidimensional de los números pseudoaleatorios generados mediante un Generador Lineal Congruencial (GLC). Esta técnica de visualización consiste en representar cada valor generado como un píxel en una matriz de 256×256 , donde la intensidad del color (en escala de grises) corresponde directamente al valor numérico producido por el generador, permitiendo así una evaluación cualitativa de la distribución de los números generados.

Los parámetros utilizados para la generación de los números fueron:

- Multiplicador (a): 1664525
- Incremento (c): 1013904223
- Módulo (m): 2^{32} (4294967296)
- Semilla inicial: 42

Estos valores se aplican en la fórmula recurrente característica de los GLC:

$$X_{n+1} = (a \cdot X_n + c) \bmod m$$

Donde cada valor generado se normaliza posteriormente al intervalo $[0,1]$ mediante la división por el módulo.

El gráfico muestra una distribución aparentemente uniforme de puntos en escala de grises, similar al “ruido blanco” estadístico. Esta característica visual es indicativa de un comportamiento deseable en generadores pseudoaleatorios, ya que sugiere:

- **Cobertura homogénea del espacio:**
Los valores generados parecen cubrir uniformemente todo el espacio bidimensional sin dejar "huecos." regiones sobrepobladas evidentes.
- **Ausencia de patrones visibles:**
No se observan estructuras repetitivas como líneas, bloques o agrupaciones que indicarían correlaciones no deseadas entre valores consecutivos.
- **Distribución equilibrada de intensidades:**
La imagen presenta una mezcla adecuada de tonos claros y oscuros, sin predominancia aparente de ciertos rangos de valores.

El análisis visual de este gráfico de ruido sugiere que el GLC implementado presenta características favorables para aplicaciones de simulación que no requieran propiedades estadísticas excepcionalmente rigurosas. Sin embargo, es importante contextualizar estos resultados:

- **Limitaciones inherentes:**

A pesar de la aparente uniformidad, los GLC son conocidos por formar estructuras geométricas regulares en espacios de dimensiones superiores (fenómeno conocido como “reticulado”), lo cual no es detectable en esta visualización bidimensional simple.

- **Período finito:**

El generador tiene un período máximo teórico de 2^{32} , aunque la calidad de la distribución puede deteriorarse antes de alcanzar este límite.

- **Complementariedad de pruebas:**

Este análisis visual debe complementarse con pruebas estadísticas formales (como los tests de Chi-cuadrado, Kolmogorov-Smirnov, entre otras) para una evaluación completa de la calidad del generador. Estas pruebas también fueron desarrolladas en este trabajo de investigación, y sus resultados serán expuestos en la siguiente sección.

En el contexto de la simulación computacional, este análisis tiene importantes implicaciones:

- **Adecuación para modelos estocásticos:**

La aparente uniformidad sugiere que el generador podría ser adecuado para modelos de simulación que requieran muestreo uniforme del espacio de estados.

- **Consideraciones para simulaciones sensibles:**

Para simulaciones donde pequeñas correlaciones puedan amplificarse, sería recomendable utilizar generadores más sofisticados como Mersenne Twister, también analizado en este informe.

- **Eficiencia computacional:**

El GLC ofrece una buena relación entre calidad aparente y eficiencia algorítmica, siendo una opción viable para aplicaciones que requieran generar grandes cantidades de números pseudoaleatorios con recursos computacionales limitados.

6.1.2. Scatter Plot en el Test de Dispersión

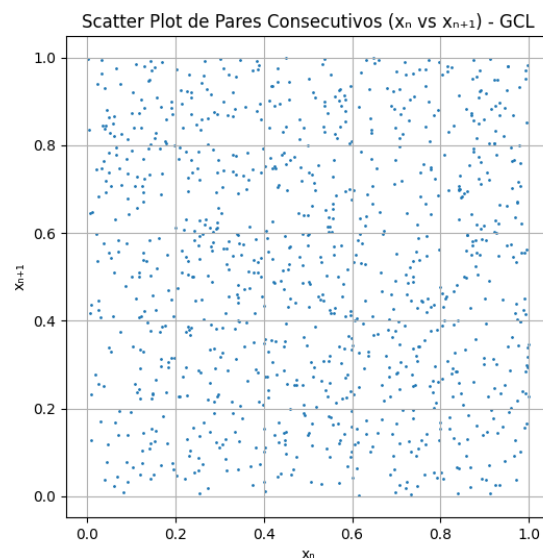


Figura 2: Gráfico de Dispersión correspondiente al Generador Pseudoaleatorio GLC

Este gráfico es un Scatter Plot de Pares Consecutivos generados por el algoritmo GCL (Generador Congruencial Lineal). Esta visualización es parte del Test de Dispersión mencionado en la Sección 4.1.1.

En este gráfico:

- El eje X (x_n) representa cada número pseudoaleatorio generado.
- El eje Y (x_{n+1}) representa el siguiente número en la secuencia.
- Cada punto azul corresponde a un par de números consecutivos (x_n, x_{n+1}).
- El rango de ambos ejes va de 0 a 1, ya que los valores están normalizados (divididos por el módulo).

Lo que podemos observar en el gráfico es una estructura en forma de rejilla, donde los puntos se distribuyen en un patrón cuadrulado, formando aproximadamente 25 regiones (5×5) visibles.

Este patrón en forma de rejilla es una característica distintiva del GCL y revela una propiedad fundamental de este generador:

- **Estructura geométrica:**
Los puntos no se distribuyen de manera completamente aleatoria, sino que caen en líneas o planos específicos dentro del espacio bidimensional.
- **Limitación del período:**
La estructura en cuadrícula está relacionada con el período máximo del generador y los parámetros elegidos (multiplicador, incremento y módulo).

La presencia de este patrón de cuadrícula indica:

- **Correlación entre valores consecutivos:**
Existe una relación matemática predecible entre un número generado y el siguiente.
- **Falta de aleatoriedad ideal:**
Un generador verdaderamente aleatorio tendría puntos distribuidos uniformemente por todo el espacio, sin formar patrones visibles.
- **Riesgos en aplicaciones:**
Este comportamiento puede ser problemático en aplicaciones que asumen independencia entre valores consecutivos.

6.1.3. Histograma 2D en el Test Serial

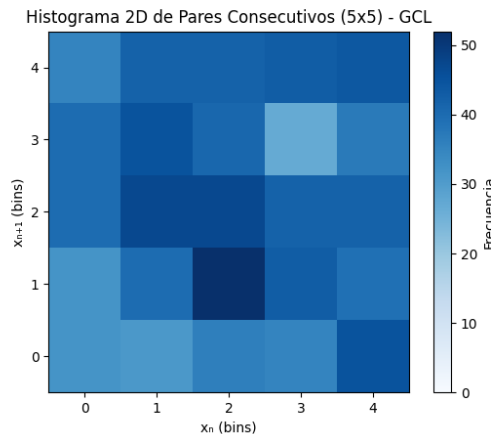


Figura 3: Histograma 2D correspondiente al Generador Pseudoaleatorio GLC

En la Figura 3 se muestra un histograma bidimensional que representa la frecuencia con la que ocurren pares consecutivos de valores (x_n, x_{n+1}) generados por el Generador Congruencial Lineal (GCL). Su propósito es evaluar la independencia entre estos números consecutivos generados por el GCL y detectar posibles patrones o estructuras que no deberían estar presentes en un generador verdaderamente aleatorio.

El espacio de números generados se divide en una grilla de 5×5 celdas (bins), y cada celda indica cuántas veces un par consecutivo cayó en esa combinación específica de valores.

La coloración del gráfico refleja la frecuencia: los tonos más oscuros indican mayor cantidad de ocurrencias. Idealmente, si el generador fuera perfectamente uniforme e independiente, se esperaría una distribución relativamente homogénea en todo el histograma. Sin embargo, la presencia de patrones o concentraciones en determinadas regiones puede ser indicio de correlaciones entre valores consecutivos o de una distribución no completamente uniforme, lo cual es característico de ciertos generadores congruenciales.

Este tipo de análisis visual permite evaluar cualitativamente la calidad del generador en términos de independencia entre números sucesivos.

Si analizamos específicamente lo que sucede con los valores generados por el GLC, se observa que algunas celdas presentan frecuencias mayores que otras, en particular la celda correspondiente a (x_n, x_{n+1}) en el bin $(2,1)$, que tiene la mayor concentración de pares. Esto indica una dependencia entre ciertos valores consecutivos generados por el GCL, lo cual sugiere una distribución no completamente uniforme.

Esta falta de homogeneidad es una limitación conocida de los generadores congruenciales lineales, especialmente cuando se analiza su comportamiento en dimensiones mayores. La presencia de zonas con menor o nula densidad refuerza la idea de que el GCL no cubre de manera equitativa el espacio de salida, lo que compromete su calidad como generador de números aleatorios en contextos donde se requiere independencia estadística entre muestras.

6.2. Generador de Multiplicación con Acarreo (MWC)

6.2.1. Gráfico de Ruido

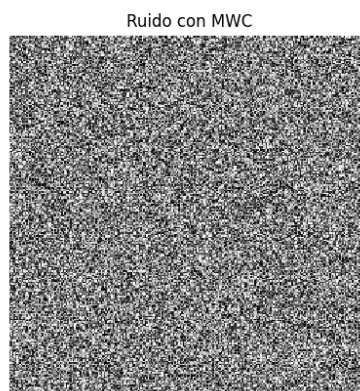


Figura 4: Gráfico de Ruido correspondiente al Generador Pseudoaleatorio MWC

Este tipo de gráficos son utilizados para representar un conjunto de números como píxeles de una imagen, de esta forma se le permite a uno verificar de forma visual la aleatoriedad del conjunto observando si hay patrones o no. En este caso, no hay patrones visibles, por ende, al ojo humano no hay aleatoriedad.

6.2.2. Scatter Plot en el Test de Dispersión

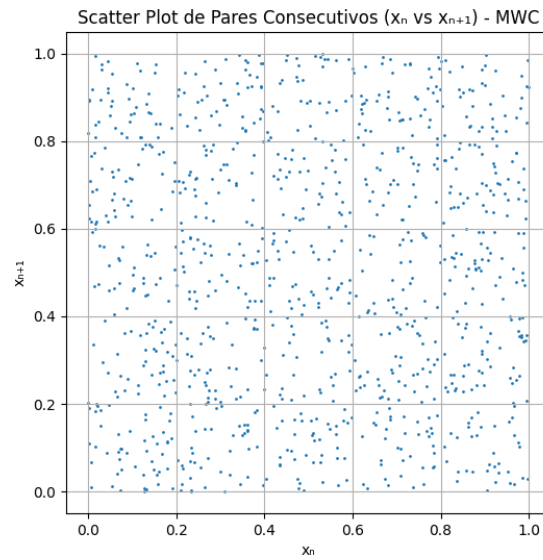


Figura 5: Gráfico de Dispersión correspondiente al Generador Pseudoaleatorio MWC

El gráfico de Scatter Plot es un gráfico de dispersión que toma pares consecutivos (X_i, X_{i+1}) para ver si existe alguna correlación o patrón entre los números del conjunto. Se usa para evaluar independencia de los números generados de forma visual, al igual que el gráfico anterior. Una vez más, no se observan patrones a simple vista, implicando la independencia de los números del conjunto.

Si hubiese algún patrón, este podría haber sido detectado si es que al observar todas las celdas, éstas tienen frecuencias muy altas o muy bajas, demostrando que hay alguna dependencia o sesgo. Como en este caso no hay concentraciones extremas o vacíos muy grandes, se puede decir que es un muy buen generador de números pseudoaleatorios el que generó este conjunto.

6.2.3. Histograma 2D en el Test Serial

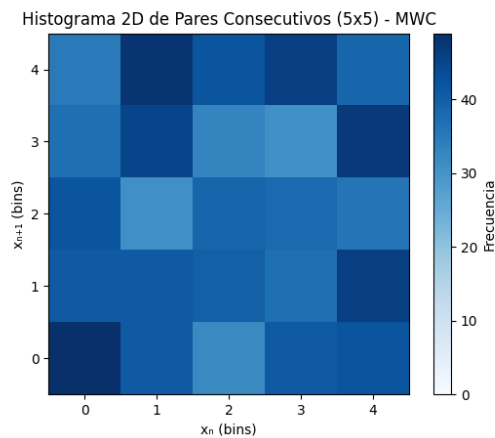


Figura 6: Histograma 2D correspondiente al Generador Pseudoaleatorio MWC

El Histograma 2D muestra la relación entre dos variables en un conjunto de datos, es por ello que se toman pares del conjunto de números y se los coloca en la gráfica. Luego, se divide la gráfica en forma de grilla (5x5 en este caso), se cuenta cuanta cantidad de puntos hay por sector y, por último, se pinta cada sector de la grilla de un color representando la frecuencia de puntos de cada uno; a mayor frecuencia, más oscuro es el azul en nuestro caso.

6.3. Generador Mersenne Twister

6.3.1. Gráfico de Ruido



Figura 7: Gráfico de Ruido correspondiente al Generador Pseudoaleatorio MT

Una vez más, así como pasó en el caso anterior, no se observan patrones al armar este tipo de gráfico. Por ende, en el conjunto de números obtenidos gracias al generador Mersenne Twister parece haber aleatoriedad.

6.3.2. Scatter Plot en el Test de Dispersión

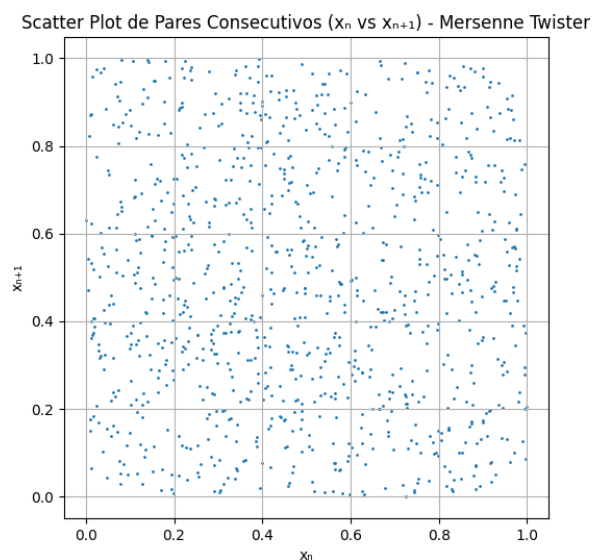


Figura 8: Gráfico de Dispersión correspondiente al Generador Pseudoaleatorio MT

Observando el gráfico 8, no se logran apreciar curvas, líneas o alguna forma reconocible como para determinar que hay patrones a la vista. De esta forma, se puede asegurar la aleatoriedad del conjunto de números.

6.3.3. Histograma 2D en el Test Serial

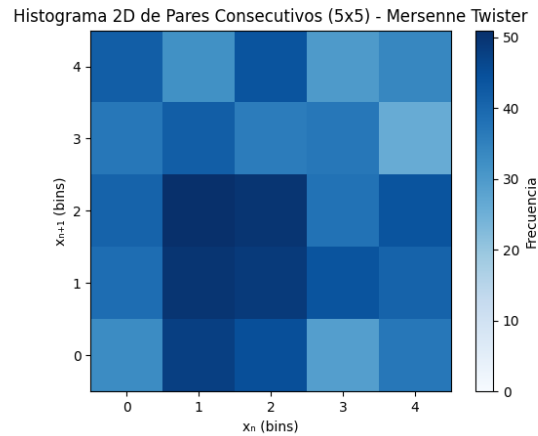
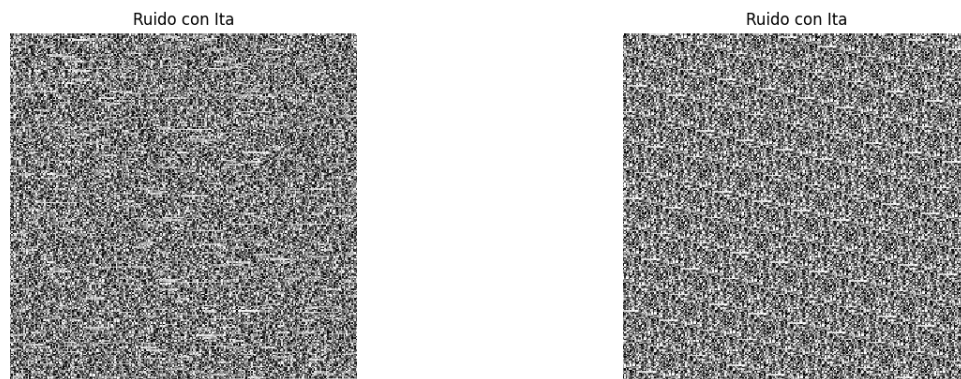


Figura 9: Histograma 2D correspondiente al Generador Pseudoaleatorio MT

El gráfico 9, muestra una distribución homogénea de los pares consecutivos, sin presencia de celdas vacías y con una cobertura adecuada de todo el espacio de valores. Se observa una leve variabilidad en las frecuencias entre las distintas celdas, con algunas zonas ligeramente más densas o más dispersas. Estas diferencias son esperables debido a las fluctuaciones estadísticas propias de una muestra finita y no indican patrones o dependencias evidentes. En consecuencia, el generador Mersenne Twister presenta un buen desempeño en el test serial 2D, evidenciando una adecuada independencia y uniformidad en la secuencia de números generada.

6.4. Generador Itamaracá (ITA)

6.4.1. Gráfico de Ruido



(a) Valor de x definido en 1.9789

(b) Valor de x definido en 2

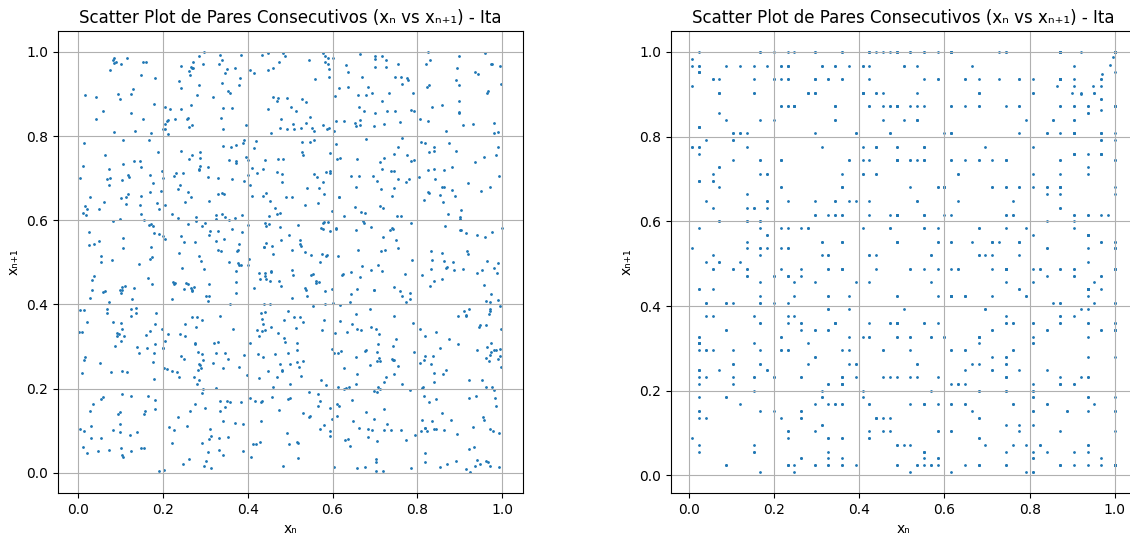
Figura 10: Gráficos de Ruido correspondientes al Generador Pseudoaleatorio ITA

Como se puede ver en la figura 10a, cuando elegimos un valor de x cercano a 2, no se aprecian patrones y el ruido es uniforme. En cambio en la figura 10b, cuando se elige que el valor de x sea igual a 2, se ven patrones a simple vista. Esto se debe a que eligiendo el valor 2, la fórmula queda:

$$FRNS = ABS[N - (Pn \cdot 2)]$$

Esto hace que el cálculo sea más predecible ya que x es un número entero, lo que reduce la calidad del generador de números pseudoaleatorios. Además, esto puede llegar a hacer que el generador caiga en ciclos pequeños rápidamente y que la sensibilidad al cambio de semillas disminuya, haciendo que secuencias distintas terminen convergiendo.

6.4.2. Scatter Plot en el Test de Dispersión



(a) Valor de x definido en 1.9789

(b) Valor de x definido en 2

Figura 11: Gráficos de Dispersión correspondientes al Generador Pseudoaleatorio ITA

En la figura 11a, se observa que los pares consecutivos representados por los puntos, están distribuidos uniformemente, esto implica que a simple vista no se puedan apreciar patrones. En cambio, en la figura 11b, se aprecia que los puntos forman líneas entre sí y no hay desorden alguno, esto se debe al valor de x elegido como se explicó anteriormente lo cual genera una falta de aleatoriedad.

6.4.3. Histograma 2D en el Test Serial

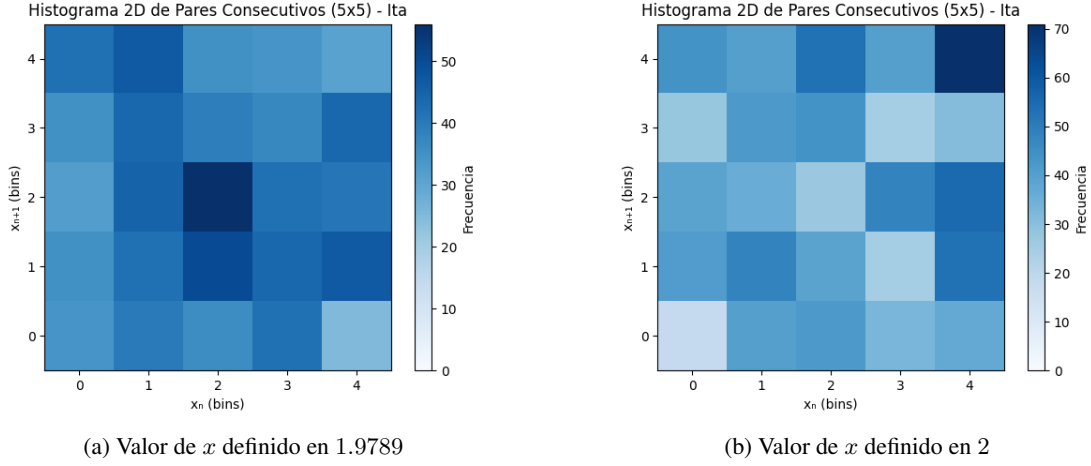


Figura 12: Histogramas 2D correspondientes al Generador Pseudoaleatorio ITA

En la figura 12, el color de las celdas representa la frecuencia de los pares (x_i, x_{i+1}) en cada celda.

En el gráfico 12a, los colores de las celdas están bien distribuidos ya que la intensidad de los colores no varía demasiado. Además, no hay acumulaciones extremadamente marcadas, aunque hay una celda con frecuencia mayor, visualmente es equilibrado. Por el contrario, en la figura 12b hay varias celdas más oscuras que otras en las cuales la frecuencia de puntos fue baja. Esto quiere decir, que la distribución está mas concentrada, con ciertas celdas que se destacan más que otras.

7. Fórmulas empleadas

Durante el desarrollo de los generadores pseudoaleatorios y los tests que comprueban su calidad, se utilizaron las fórmulas descritas a continuación.

7.1. Fórmulas empleadas por los Generadores de Números Pseudoaleatorios

7.1.1. Generador Lineal Congruencial (GLC)

Durante la implementación del Generador Lineal Congruencial (GLC), se empleó la siguiente fórmula:

Fórmula de recurrencia del GLC

$$X_{n+1} = (a \cdot X_n + c) \text{ mód } m$$

donde:

- X_n : valor generado en la iteración n .
- a : multiplicador.
- c : incremento.
- m : módulo.
- X_0 : semilla inicial.

Esta fórmula permite generar una sucesión determinista de números enteros en el intervalo $[0, m - 1]$, dependiendo del valor inicial y los parámetros elegidos.

7.1.2. Generador de Multiplicación con Acarreo (MWC)

Durante la implementación del Generador de Multiplicación con Acarreo (MWC), se utilizaron las siguientes fórmulas:

Fórmula de actualización del estado interno

El generador MWC produce números pseudoaleatorios a partir de una combinación entre una semilla, un acarreo y un multiplicador. En cada iteración, se realiza el siguiente cálculo intermedio:

$$T_n = a \cdot X_n + C_n$$

donde:

- T_n : valor total intermedio,
- X_n : valor actual de la semilla en la iteración n ,
- C_n : valor del acarreo en la iteración n ,
- a : multiplicador.

A partir de este valor intermedio, se actualiza el estado del generador de la siguiente forma:

$$X_{n+1} = T_n \text{ mód } m$$

$$C_{n+1} = \left\lfloor \frac{T_n}{m} \right\rfloor$$

donde:

- X_{n+1} : nuevo valor de la semilla,
- C_{n+1} : nuevo valor del acarreo,
- m : módulo.

Estas fórmulas garantizan que tanto el valor generado como el acarreo queden acotados por el módulo definido, asegurando la estabilidad de la secuencia generada.

7.1.3. Generador Mersenne Twister (MT)

Durante la implementación del Generador Mersenne Twister (MT), se utilizó una función pseudoaleatoria integrada del lenguaje Python, que internamente se basa en el algoritmo Mersenne Twister.

A diferencia de otros generadores como el GLC o el MWC, el Mersenne Twister no se implementa manualmente mediante una fórmula recursiva explícita, sino que se invoca a través de una función de biblioteca que ya encapsula la complejidad del algoritmo.

Generación de números enteros

El valor entero pseudoaleatorio X_n se genera utilizando:

$$X_n \in \{0, 1, 2, \dots, m\}$$

donde:

- X_n : valor entero generado en la iteración n ,
- m : límite superior (módulo).

El generador retorna un valor entero uniforme dentro del intervalo cerrado $[0, m]$, utilizando internamente una semilla automática o definida por el entorno de ejecución del sistema.

7.1.4. Generador Itamaracá (ITA)

Durante la implementación del Generador Itamaracá (ITA), se utilizaron dos fórmulas principales que conforman su lógica de generación. Este generador se basa en diferencias sucesivas entre semillas y una constante de multiplicación cercana a 2, con el objetivo de producir una secuencia pseudoaleatoria acotada.

Estado intermedio (Proceso P_n)

$$P_n = |S_2 - S_0|$$

donde:

- S_0, S_1, S_2 : semillas utilizadas en cada iteración,
- P_n : valor intermedio en el paso n .

Esta fórmula calcula la diferencia absoluta entre dos de las semillas activas del generador, que se actualizan en cada ciclo.

Fórmula de generación final

$$X_n = |N - (P_n \cdot x)|$$

donde:

- N : valor máximo permitido (rango superior),
- x : constante multiplicativa cercana a 2 (por ejemplo, $x = 1.9789$),
- X_n : valor pseudoaleatorio entero generado en la iteración n .

El resultado de esta operación se convierte en la nueva semilla y se utiliza para calcular los valores siguientes, reemplazando cíclicamente las semillas anteriores.

7.1.5. Normalización al intervalo $[0, 1)$

Todos los generadores de números pseudoaleatorios descritos en esta sección producen inicialmente valores enteros acotados dentro de un rango definido por su módulo, semilla o parámetros internos.

Para poder utilizar estos valores en pruebas estadísticas de uniformidad, visualizaciones gráficas y otros análisis, es necesario convertirlos a números reales en el intervalo $[0, 1)$.

La fórmula utilizada para dicha normalización es la siguiente:

$$U_n = \frac{X_n}{R}$$

donde:

- X_n : valor entero generado por el generador en la iteración n ,
- R : valor de referencia para cada generador, que puede ser el módulo, el rango máximo o un valor fijo dependiendo del caso.

Esta operación asegura la compatibilidad entre generadores a la hora de comparar sus secuencias y aplicar los mismos criterios de evaluación estadística.

7.2. Fórmulas empleadas por los Tests de los Generadores

7.2.1. Test de Dispersión (Scatter Plot)

El Test de Dispersión, también conocido como *Scatter Plot*, es una herramienta gráfica utilizada para evaluar visualmente la presencia de patrones o estructuras deterministas en una secuencia de números pseudoaleatorios. Este test forma parte de las pruebas empíricas de aleatoriedad y se basa en el análisis de la independencia entre valores consecutivos de una secuencia generada.

Formación de pares consecutivos

A partir de una secuencia de $n + 1$ números pseudoaleatorios normalizados $[x_0, x_1, x_2, \dots, x_n]$, se construyen n pares consecutivos de la forma:

$$(x_0, x_1), (x_1, x_2), \dots, (x_{n-1}, x_n)$$

Representación gráfica

Cada uno de estos pares se representa como un punto en el plano cartesiano, donde el eje horizontal (x) corresponde a x_n y el eje vertical (y) a x_{n+1} . La gráfica resultante permite observar visualmente la distribución de los pares y detectar posibles alineamientos, acumulaciones o estructuras geométricas no deseadas.

Interpretación

- Una distribución uniforme y dispersa de puntos en todo el plano indica una alta independencia entre valores consecutivos, lo que sugiere buena aleatoriedad.
- La aparición de patrones como líneas, rejillas, curvas o concentraciones localizadas puede revelar correlaciones internas y dependencia estadística entre los valores, lo cual compromete la calidad del generador.

Consideraciones

Aunque el test de dispersión no proporciona un valor numérico como resultado (es puramente visual), es una herramienta útil para detectar rápidamente deficiencias evidentes en la independencia de la secuencia. Por esta razón, suele emplearse como complemento preliminar antes de aplicar pruebas estadísticas más rigurosas.

7.2.2. Test Serial

Durante el desarrollo del Test Serial, se utilizaron las siguientes fórmulas:

Estadístico Chi-cuadrado

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

donde:

- O_i : frecuencia observada.
- E_i : frecuencia esperada.

Esta fórmula se usa para medir cuán lejos están los datos observados de lo que se esperaría bajo una hipótesis nula. Es decir, sirve para evaluar si una distribución observada difiere significativamente de una distribución teórica esperada. En este caso verifica si hay dependencia entre pares consecutivos de números. Se compara la distribución de pares (x_i, x_{i+1}) con la distribución esperada si fueran independientes.

7.2.3. Test de Autocorrelación

Autocorrelación (r)

$$r = \frac{\sum_{i=1}^{n-lag} (x_i - \bar{x})(x_{i+lag} - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

donde:

- x_i : valor en la posición i.
- $\bar{x}$: media de todos los valores.
- lag : desfase.
- n : cantidad total de datos.

Esta fórmula mide cuánto se parecen los valores separados por cierta distancia (lag). Si r es cercano a 0, no hay correlación, es decir, hay una buena señal de independencia.

Error estándar del estimador

$$SE = \sqrt{\frac{1 + 2 * r^2}{n}}$$

Esta fórmula da una medida de la variabilidad esperada del valor de autocorrelación si no hubiera correlación (hipótesis nula). Mientras más grande sea n, más pequeño será el error estándar.

Estadístico z

$$z = \frac{r}{SE}$$

Esta fórmula compara el valor de autocorrelación con el error estándar.

Valor crítico

$$z_{critico} = \phi^{-1}(1 - \alpha/2)$$

donde:

- ϕ^{-1} : función inversa de la distribución normal estándar (también conocida como cuantil o percentil).
- α : nivel de significancia.

Esta expresión representa un umbral calculado de acuerdo con el nivel de significancia (α), y nos dice cuál es el límite a partir del cual podemos rechazar la hipótesis nula.

7.2.4. Test Chi-Cuadrado

Frecuencia esperada por intervalo

$$E_i = \frac{n}{k}$$

donde:

- n : cantidad total de datos generados.
- k : cantidad de intervalos.
- E_i : frecuencia esperada en el intervalo i .

Estadístico Chi-cuadrado

$$x^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

Esta fórmula mide cuánto se desvían las observaciones respecto a lo esperado. Si los valores están muy alejados, el generador es uniforme.

7.2.5. Test Kolmogorov-Smirnov

Función de distribución empírica

$$F_n(x) = \frac{\#\{x_i \leq x\}}{n}$$

Estadístico D_n

$$D_n = \sup_x |F_n(x) - F(x)|$$

Cálculo del valor de p

$$p = P(\sqrt{n}D_n \geq \sqrt{nd}) = 2 \sum_{k=1}^{\infty} (-1)^{k-1} e^{-2k^2nd^2}$$

7.2.6. Test de la Media

Media muestral

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

Calcula el promedio de todos los números en la muestra. Es una estimación de la media verdadera de la población.

Varianza teórica

$$\sigma^2 = \frac{1}{12}$$

Usa la varianza teórica de la distribución uniforme $U[0,1]$, que siempre vale $1/12$.

Error estándar

$$SE = \sqrt{\frac{\sigma^2}{n}} = \sqrt{\frac{1}{12n}}$$

Calcula el error estándar, que mide cuánto puede variar la media muestral simplemente por azar, dependiendo del tamaño de la muestra n .

Estadístico de prueba

$$Z = \frac{\bar{X} - \mu_0}{SE}$$

Mide cuán lejos está la media muestral de la media teórica ($\mu_0 = 0.5$), en unidades del error estándar.

Valor crítico para el intervalo de confianza (bilateral)

$$z_{critico} = \phi^{-1}(1 - \alpha/2)$$

Calcula el valor crítico que define el umbral de rechazo de la hipótesis nula.

8. Conclusiones

Mediante la realización de este trabajo teórico-práctico sobre generadores pseudoaleatorios, pudimos analizar y contrastar el comportamiento de distintos métodos de generación de números aleatorios, tanto desde una perspectiva algorítmica como estadística.

A lo largo del trabajo se implementaron y evaluaron distintos generadores, como el lineal congruencial (GLC), el de multiplicación con acarreo (MWC), el Mersenne Twister (MT) y el Itamaracá (ITA). En cada caso, se observaron sus características particulares en términos de distribución, independencia y uniformidad, aspectos fundamentales para determinar la calidad de un generador.

Nuestros resultados muestran que, si bien todos los generadores producen secuencias que aparentan ser aleatorias, no todos presentan la misma calidad estadística. Algunos generadores como el congruencial lineal pueden sufrir de deficiencias como ciclos cortos o patrones repetitivos, mientras que otros más complejos como Mersenne Twister logran una mayor aleatoriedad aparente.

Estos experimentos y análisis nos permitieron visualizar en la práctica cómo las propiedades teóricas se manifiestan en las secuencias generadas. Pudimos comprobar que la correcta implementación de los parámetros (como el módulo, multiplicador, incremento y semilla) es esencial para garantizar un buen comportamiento del generador.

Asimismo, se destacó la importancia de aplicar pruebas estadísticas y visualizar gráficamente las secuencias para evaluar su calidad y uniformidad en aplicaciones que requieren aleatoriedad, como experimentos computacionales o procesos de simulación.

Como conclusión final, este trabajo nos permitió comprender en profundidad cómo operan los generadores pseudoaleatorios y que tan cruciales son sus características internas para asegurar resultados confiables en experimentos computacionales. A través de este enfoque práctico y analítico, se comprueba la importancia de una buena elección del generador al trabajar con fenómenos que involucran incertidumbre o simulación aleatoria.

Referencias

- Calculator.net. (2025). *Generador de Números Aleatorios*. Calculator.net. Consultado el 8 de mayo de 2025, desde <https://www.calculator.net/random-number-generator.html>
- DilliHangrae. (2024). *AutoCorrelation Test*. Medium. Consultado el 8 de mayo de 2025, desde <https://medium.com/@dillihangrae/autocorrelation-test-ca93efd741c8>
- Pereira, D. H. (2022). *Itamaracá: Nueva Manera Sencilla de Generar Números Pseudoaleatorios*. SlideShare. Consultado el 9 de mayo de 2025, desde <https://www.slideshare.net/DHPereira1/itamarac-nueva-manera-sencilla-de-generar-nmeros-pseudoaleatorios>
- Pulido Cayuela, M. A. (2009). *Generación de números aleatorios - Sección 1.3: Tests para la comprobación de la uniformidad y la aleatoriedad*. Universidad de Murcia. Consultado el 8 de mayo de 2025, desde <https://webs.um.es/mpulido/miwiki/lib/exe/fetch.php?id=amio&cache=cache&media=wiki:simt1b.pdf>
- QuantStart. (2025). *Linear Congruential Generators in Python*. QuantStart. Consultado el 8 de mayo de 2025, desde <https://www.quantstart.com/articles/linear-congruential-generators-in-python/>
- Tereom. (2018). *Números Pseudoaleatorios*. Tereom. Consultado el 8 de mayo de 2025, desde <https://tereom.github.io/est-computacional-2018/numeros-pseudoaleatorios.html>
- UniverTools. (2017). *Prueba chi-cuadrada para números aleatorios*. YouTube. Consultado el 8 de mayo de 2025, desde <https://www.youtube.com/watch?v=tsYM0un-q3w>
- Watermarkero. (2017). *Estegoanálisis usando Python, prueba Chi cuadrada del método LSB*. YouTube. Consultado el 8 de mayo de 2025, desde <https://www.youtube.com/watch?v=Wdn1blcLVw>
- Wikipedia contributors. (2025a). *Generador lineal congruencial*. Wikipedia. Consultado el 8 de mayo de 2025, desde https://es.wikipedia.org/wiki/Generador_lineal_congruencial
- Wikipedia contributors. (2025b). *Multiply-with-carry pseudorandom number generator*. Wikipedia. Consultado el 8 de mayo de 2025, desde https://en.wikipedia.org/wiki/Multiply-with-carry_pseudorandom_number_generator